Parcial 2 - Transcripcion completa

Teoría de la Información — Parcial 2: transcripción completa de incisos y resoluciones

> Fuente: todo el contenido de la carpeta `Teoria de la Información/Parcial 2/Parciales 2/`. Las convenciones (polinomios, matrices, ISBN, Reed–Solomon) son las de la cátedra, tal como están en `Parcial 2/PDF Practicos/` y en el Apunte Maestro del Parcial 2.
> Marcas usadas: **[Corrección]** = comentario del docente en el escaneo, o error de la resolución de un alumno que detecté al verificarla; **[Nota]** = aclaración mía (algo ilegible, ambiguo o una errata del enunciado); **[Completado]** = resolución que el archivo no traía (los parciales viejos solo tienen enunciado) y agregué yo.
> Todos los cálculos (divisiones polinómicas, pesos, matrices con $G \cdot H^T = 0$, descifrados, entropías, ISBN, inversos modulares) fueron recalculados por programa; los comentarios del docente de la alumna Arce están transcriptos tal cual. De la última página del PDF de Arce (una foto de un documento personal) no se transcribe nada.

Índice de archivos y qué contiene

Archivo	Contenido
`Parcial 2/PARCIAL 2 TEORÍA DE LA INFORMACIÓN.pdf` y `.docx`	**Parcial del 17/11/2025, Tema 1**, resuelto por la alumna Bulián Melani (el `.docx` es el mismo texto editable). Ver PARTE 1
`Parcial N2 de Teoria de la Informacion Tema 1 Contreras Matias.docx` (estaba en Descargas, no en la carpeta)	**El mismo parcial del 17/11/2025, Tema 1**, resuelto por otro alumno (Contreras Matías); resolución incompleta. Ver PARTE 1 bis
`parciales viejos/Johana Arce Parcial N°2 … CORREGIDO-APROBADO.pdf` (6 págs.)	**Parcial del 15/12/2020** resuelto a mano por la alumna Arce, con comentarios del docente y nota **7,2**. Págs. 1–5 = enunciado y resolución; pág. 6 = foto de documento personal (no se transcribe). Ver PARTE 2
`parciales viejos/PARCIAL2TI.doc.pdf`	**Parcial del 29/11/2010** (enunciado, más una encuesta de cátedra que no es del parcial). Ver PARTE 3
`parciales viejos/Parcial 22012-TI.doc.pdf`	**Parcial del 21/11/2012, Temas 1 y 2** (enunciados). Ver PARTE 4
`parciales viejos/Parcial 2 - TI 2013.doc.pdf`	**Parcial del 12/11/2013, Temas 1 y 2** (enunciados; el PDF repite cada tema dos veces). Ver PARTE 5
`parciales viejos/Preguntas Teóricas parcial 2 Teoría de la información.pdf`	Preguntas teóricas de cripto y control de error con respuestas de alumno. Ver PARTE 6
`parciales viejos/Parcial1TIpdf (1).pdf`	**No es del Parcial 2**: es el enunciado de los Temas 1–4 del Parcial 1 (idéntico byte a byte a `Parcial 1/Parciales 1/Parcial1TIpdf_1.pdf`, ya transcripto en `Parcial 1 - Transcripcion completa.md`). Descartado por repetido

Los parciales de 2010, 2012 y 2013 vienen **sin resolución**: todo lo que figura como resolución en esas partes es **[Completado]**.

PARTE 0 — Recetas y convenciones (cómo se resuelve cada tipo de inciso)

0.1 Convenciones

- **Cadena de bits ↔ polinomio ("mayores potencias a la derecha")**: $b_0\ b_1\ \dots\ b_{n-1}$ es $b_0 + b_1x + \dots + b_{n-1}x^{n-1}$. Para dividir a mano se escribe con la mayor potencia a la izquierda y al terminar se vuelve a la convención del enunciado.
- **Cíclicos**: n = largo de palabra, $k = \text{grado de } h(x)$ (bits de información), $n-k$ chequeadores, $g(x) = (x^n + 1)/h(x)$ (grado $n-k$). $h(x)$ es el polinomio de **chequeo** que da el enunciado, no el generador.
- **Palabras válidas** = múltiplos de $g(x)$; sistemática: $V(x) = x^{n-k}\cdot i(x) + r(x)$ con $r = x^{n-k}\cdot i \bmod g$ (los chequeadores van en las menores potencias y la información en las mayores). $G = [P \mid I_k]$ (fila i de $P = x^{n-k+i} \bmod g$) y $H = [I_{n-k} \mid P^T]$, con $G\cdot H^T = 0$.
- **Capacidad**: $d_{min} \geq 2t + 1$ para corregir t errores; como detector puro detecta $d_{min} - 1$; corrige t y detecta s a la vez si $d_{min} \geq t + s + 1$.
- **Atajo**: $d_{min} \leq$ cantidad de términos de $g(x)$ (porque $g(x)$ es una palabra del código); en los tres códigos de abajo da igualdad.
- **Redundancia (cátedra) = N/K**.

0.2 Los tres códigos cíclicos que se repiten en los parciales

Código	Aparece en	`h(x)`	`g(x)` (bits, mayor potencia a la derecha)	`dmin`	Pesos de las palabras no nulas	Corrige / detecta	`N/K`	Ráfaga con `L` códigos entrelazados
(12, 3)	2010, 2013 T2, 2020	$1 + x + x^2 + x^3$	$1 + x + x^4 + x^5 + x^8 + x^9$ (`1100110011`)	**6**	6 y 12	corrige 2 y detecta 3 más; como detector, 5	12/3 = 4	`2L` (L=7 → 14; L=20 → 40)
(14, 4)	2012 T2, 2013 T1, 2025	$1 + x + x^2 + x^4$	$1 + x + x^3 + x^7 + x^8 + x^{10}$ (`11010001101`)	**6**	6, 8 y 14	corrige 2 y detecta 3 más; como detector, 5	14/4 = 3,5	`2L` (L=20 → 40; L=25 → 50)
(15, 4)	2012 T1	$1 + x + x^4$	$1 + x + x^2 + x^3 + x^5 + x^7 + x^8 + x^{11}$ (`111101011001`)	**8**	todas 8	corrige 3 y detecta 4 más; como detector, 7	15/4	`3L` (L=20 → 60)

Matrices sistemáticas $G = [P \mid I_k]$ y $H = [I_{n-k} \mid P^T]$ y chequeadores (`Cj` = XOR de los `Ii` indicados; `I1 I2 …` son los bits de información de menor a mayor potencia). Todas verificadas con $G \cdot H^T = 0$:

(12, 3) — `P` = `110011001 / 101010101 / 100110011`

```
C1 = I1⊕I2⊕I3   C2 = I1   C3 = I2   C4 = I3   C5 = I1⊕I2⊕I3
C6 = I1   C7 = I2   C8 = I3   C9 = I1⊕I2⊕I3

G (3×12)            H (9×12)
110011001 100       100000000 111
101010101 010       010000000 100
100110011 001       001000000 010
                    000100000 001
                    000010000 111
                    000001000 100
                    000000100 010
                    000000010 001
                    000000001 111
```

(14, 4) — `P` = `1101000110 / 0110100011 / 1110010111 / 1010001101`

```
C1 = I1⊕I3⊕I4   C2 = I1⊕I2⊕I3   C3 = I2⊕I3⊕I4   C4 = I1   C5 = I2
C6 = I3   C7 = I4   C8 = I1⊕I3⊕I4   C9 = I1⊕I2⊕I3   C10 = I2⊕I3⊕I4

G (4×14)               H (10×14)
1101000110 1000        1000000000 1011
0110100011 0100        0100000000 1110
1110010111 0010        0010000000 0111
1010001101 0001        0001000000 1000
                       0000100000 0100
                       0000010000 0010
                       0000001000 0001
                       0000000100 1011
                       0000000010 1110
                       0000000001 0111
```

(15, 4) — `P` = `11110101100 / 01111010110 / 00111101011 / 11101011001`

```
C1 = I1⊕I4        C2 = I1⊕I2⊕I4     C3 = I1⊕I2⊕I3⊕I4   C4 = I1⊕I2⊕I3
C5 = I2⊕I3⊕I4    C6 = I1⊕I3        C7 = I2⊕I4         C8 = I1⊕I3⊕I4
C9 = I1⊕I2        C10 = I2⊕I3       C11 = I3⊕I4

G (4×15)                H (11×15)  = [I11 | P^T]
11110101100 1000        (cada fila j: un 1 en la posición j y, a la derecha,
01111010110 0100         los bits de la columna j de P; p. ej. fila 1 = 10000000000 1001)
00111101011 0010
11101011001 0001
```

0.3 Receta: validar, detectar y corregir una palabra recibida `R(x)` (cíclicos)

1. Dividir `R(x)` por `g(x)` en GF(2) (restar = XOR). El **resto es el síndrome**: resto 0 → válida.
2. Si no es 0: el síndrome coincide con el de un patrón de error `e(x)` de peso ≤ `t` (se prueba con errores simples $x^i \bmod g$, y si no coincide con ninguno, con sumas de 2 y de 3).
3. Corregida: `R(x) + e(x)`; comprobar que el resto da 0.
4. Si el síndrome no coincide con ningún patrón de peso ≤ `t`, el error se **detecta pero no se corrige** (se pide retransmisión).
5. Atajo útil: si `grado(R) < n−k`, el resto es el propio `R(x)` (ya está "reducido").
6. **Ráfaga corregible** con `L` códigos entrelazados: `L·t`. Dada una ráfaga de longitud `B`, se necesita `L = ⌈B/t⌉`.

0.4 ISBN (códigos pesados, base prima)

- **ISBN-10 (base 11)**: pesos `10, 9, …, 1`; válido ⇔ `Σ dígito·peso ≡ 0 (mod 11)`; el último símbolo puede ser `X = 10`. Dígito faltante `x` con peso `w`: `w·x ≡ −(suma de los demás) (mod 11)`. `dmin = 2` (detecta 1 error, no corrige).
- **ISBN-13 (base 10)**: pesos `1, 3, 1, 3, …`; válido ⇔ `Σ ≡ 0 (mod 10)`; dígito de control = `(10 − (suma de los 12 primeros mod 10)) mod 10`.
- **Por qué base prima**: en $\mathbb{Z}_p$ todo elemento no nulo tiene inverso, así el dígito faltante es **único** (decodificación unívoca) y se detectan errores simples y permutaciones de dígitos.

0.5 Sustitución afín

`C = (a·M + b) mod n`; $M = a^{-1}\cdot(C - b) \bmod n = a^{-1}\cdot C + b'$ con $b' = -a^{-1}\cdot b \bmod n$. Condición: `gcd(a, n) = 1`. $a^{-1} = a^{\phi(n)-1} \bmod n$ (Euler) o Euclides extendido. $\phi(p) = p - 1$ si `p` es primo; si no, $\phi(n) = n\cdot\Pi(1 - 1/p)$. El tamaño `n` y el **orden de los símbolos** los da el enunciado.

0.6 Cifrador compuesto (Vigenère + transposición) y entropías por etapa

- Si el enunciado dice "primero A y luego B", para descifrar se deshace **primero B y después A**.
- **Vigenère**: `C = (M + K) mod n` con la clave repetida; se descifra `M = (C − K) mod n`. Con la numeración `A = 1 … Z = 26` (2010) el resultado se escribe en `1…26` (0 ≡ 26); con `A = 0` el mismo cálculo queda **corrido una letra**.
- **Transposición por matriz**: el claro se escribe **por filas** en una matriz de `f × c` (`c` = largo de la clave) y se lee **por columnas**; la convención que da texto con sentido (verificada en el parcial de 2012) es *"la columna j se lee en la posición k_j"*.
- **Entropía de orden 0** (modelo independiente): $H = \Sigma\, p_i\cdot\log_2(1/p_i)$, $Hmax = \log_2 q$, redundancia $R = 1 - H/Hmax$.
- **Qué orden modifica cada algoritmo**: la transposición **no** modifica H_0 (mismos símbolos, mismas frecuencias) pero sí las de orden ≥ 1 (rompe las dependencias entre vecinos); Vigenère **sí** modifica H_0 (un mismo símbolo se cifra distinto según la posición, las frecuencias se aplanan) y todas las superiores; César/afín/monoalfabética no modifican ningún orden (es una relabelación biyectiva).

0.7 Imágenes (lo que aparece en los parciales)

- **Máscaras de recta** (3×3, se aplica cada máscara y gana la de mayor respuesta en valor absoluto): horizontal `[−1 −1 −1; 2 2 2; −1 −1 −1]`, vertical `[−1 2 −1; −1 2 −1; −1 2 −1]`, 45° `[−1 −1 2; −1 2 −1; 2 −1 −1]`, 135° `[2 −1 −1; −1 2 −1; −1 −1 2]`.
- **Filtro de media aritmética 3×3**: el centro pasa a ser el promedio de las 9 posiciones (suma/9). **Mediana**: se ordenan los 9 valores y el centro pasa a ser el quinto (filtro no lineal; elimina picos aislados).
- **Vectorial vs. bitmap**: la vectorial se describe con figuras matemáticas (puntos, curvas, rellenos) y escala sin pérdida; la bitmap es una matriz de píxeles, ocupa más y se pixela al ampliar.
- **RLE de PCX**: una racha de `r` repeticiones (`r ≤ 63`) se guarda como el byte `0xC0 + r` seguido del valor; un valor suelto `< 192` se guarda tal cual; un valor `≥ 192` necesita siempre el byte de escape `0xC1` antes del valor, aunque aparezca una sola vez.

0.8 Reed–Solomon

$q = 2^m$ (tamaño del campo `GF(q)`, `m` = grado del polinomio primitivo del campo = bits por byte), `n = q − 1` bytes por palabra, bytes de chequeo `n − k = 2t`, `t` = bytes corregibles; con `L` códigos entrelazados se corrige una ráfaga de `L·t` bytes.

PARTE 1 — Parcial 17/11/2025, Tema 1 (`PARCIAL 2 TEORÍA DE LA INFORMACIÓN.pdf`, alumna Bulián Melani)

Enunciado y resolución del alumno + verificación

Ejercicio 1 — Códigos de bloques lineales. Códigos cíclicos

> Sea el código (14, 4, dmin) cuyo espacio de palabras válidas lo permite construir el siguiente polinomio cíclico: `G(x) = 11010001101` (mayores potencias a la derecha). Grado de G = n − k = 10; $g(x) = 1 + x + x^3 + x^7 + x^8 + x^{10}$.

(Es el código **(14, 4)** de la tabla 0.2.)

Inciso 1.a) Encuentre la capacidad de código

Resolución del alumno. Como es lineal, `dmin = min{peso de la palabra código no nula}`. Genera las $2^4 = 16$ palabras, calcula el peso de las 15 no nulas, obtiene `dmin = 6`, `t = ⌊(6−1)/2⌋ = 2` y detecta hasta `dmin − 1 = 5` errores.

Su tabla de pesos (info `i(x)` → peso): `0001→6, 0010→7, 0100→8, 1000→7, 0011→8, 0101→7, 0110→6, 0111→7, 1001→6, 1010→8, 1011→9, 1100→7, 1101→8, 1110→7, 1111→8`.

[Corrección] El resultado final es correcto (`dmin = 6`, `t = 2`, detecta 5; como corrector corrige 2 y detecta 3), pero **la tabla de pesos es incorrecta**: de las 15 filas solo 4 coinciden (`0001`, `0011`, `1010`, `1111`). Los pesos reales (`V = i(x)·g(x)`, bits de menor a mayor potencia) son:

info (bits, 0 primero)	palabra	peso	info	palabra	peso
1000	11010001101000	6	1001	11001011100101	8
0100	01101000110100	6	0101	01110010111001	8
1100	10111001011100	8	1101	10100011010001	6
0010	00110100011010	6	0011	00101110010111	8
1010	11100101110010	8	1011	11111111111111	**14**
0110	01011100101110	8	0111	01000110100011	6
1110	10001101000110	6	1111	10010111001011	8
0001	00011010001101	6			

Pesos posibles: **6, 8 y 14** (nunca 7 ni 9). `dmin = 6`.

Inciso 1.b) Encuentre las matrices generadora y chequeadora

Resolución del alumno. `G` (4×14): `g` rellenada con ceros hasta 14 bits (`11010001101000`) y corrida cíclicamente una posición por fila:

```
11010001101000
01101000110100
00110100011010
00011010001101
```

Para `H`: `h(x) = (x¹⁴ − 1)/g(x) = 1 + x + x² + x⁴` → bits `11101`, rellena hasta 14 y arma `H` (10×14) corriendo la cadena `11101` una posición por fila.

[Corrección] La `G` es correcta. La `H` **no**: usó `h(x)` sin invertir (`11101…`), pero las filas de `H` deben ser el polinomio **recíproco** de `h(x)`, `x⁴·h(1/x) = 1 + x² + x³ + x⁴` → `10111`. Con la `H` de la alumna, `G·Hᵀ ≠ 0` (por ejemplo, la palabra `g = 11010001101000` contra la tercera fila `00111010000000` da `1`, no `0`). La `H` correcta (10×14) es:

```
10111000000000
01011100000000
00101110000000
00010111000000
00001011100000
00000101110000
00000010111000
00000001011100
00000000101110
00000000010111
```

(verificado: `G·Hᵀ = 0`; en forma sistemática ver la tabla 0.2.)

Inciso 1.c) ¿Es válida `R(x) = 11110001101000`?

`R(x) = 1 + x + x² + x³ + x⁷ + x⁸ + x¹⁰`. Se divide por `G(x)` (14 bits: `11010001101000`); `R ⊕ G = 00100000000000`, es decir resto `x² ≠ 0` → **no es válida**.

Inciso 1.d) Detectar y corregir el error

El resto `x²` es el síndrome de un **error simple en la posición** `x²` (coincide con `x² mod g`, y `2 < n−k = 10`). Se cambia ese bit: `R(x) + x² = g(x)` → `11010001101000`, resto 0 → válida (es la palabra para `i(x) = 1`).

[Corrección] La alumna escribió "R(x) corregida = `1110001101000`" (13 bits, falta uno): lo correcto es `11010001101000` (14 bits), que es el que luego usa en la división.

Inciso 1.e) Si se entrelazan 25 de esos códigos, ¿cuál es la capacidad de corrección en ráfagas?

Sin entrelazar corrige $t = 2$ errores por palabra de 14 bits. Con $L = 25$, una ráfaga de longitud B se reparte entre los 25 bloques ($\lceil B/25 \rceil$ errores por bloque) y hace falta $\lceil B/25 \rceil \le 2 \rightarrow$ **$B \le 25 \cdot 2 = 50$ bits**.

Ejercicio 2 — Códigos ISBN

> Sea el siguiente código ISBN13 `978-1-80056-662-3`, verifique si el mismo es válido o no. Si no lo fuera, ¿es posible corregirlo? Puede usar cryptool2.

Resolución del alumno. Pesos $1, 3, 1, 3, \ldots$: productos $9, 21, 8, 3, 8, 0, 0, 15, 6, 18, 6, 6, 3$, suma 103; $103 \bmod 10 = 3 \neq 0 \rightarrow$ **no es válido**.

"Es posible corregirlo": si se asume que el equivocado es el dígito verificador, se busca $?$ en `978-1-80056-662-?`; los 12 primeros dígitos suman 100 y $(100 + 1 \cdot ?) \bmod 10 = 0 \rightarrow ? = 0 \rightarrow$ **978-1-80056-662-0 es válido**.

[Nota] Verificado: suma de los 12 primeros = 100 → control correcto 0. Pero la corrección solo vale **bajo ese supuesto**: el código ($dmin = 2$) detecta que hay un error y no sabe en qué dígito está. Como los pesos 1 y 3 son invertibles módulo 10, para **cada una** de las 13 posiciones existe exactamente un dígito que haría válido el código: hay 13 "correcciones" posibles y no se puede elegir una sin información extra.

Ejercicio 3 — Arreglos geométricos

> Dado un código rectangular, donde cada fila es una palabra Hamming, y existen paridades por filas y por columnas. Encontrar la dmin del código expresando la capacidad del mismo e indicar de qué manera se lograrían reconocer y corregir los errores.

Esquema del enunciado:

```
C10  C20  I10  BPP0
C11  C21  I11  BPP1
C12  C22  I12  BPP2
BPP3 BPP4 BPP5
```

Resolución del alumno. Para un código producto de uno en filas con distancia d_1 y uno en columnas con d_2: $dmin = d_1 \cdot d_2$. Filas: Hamming, $d_1 = 3$; columnas: paridad simple, $d_2 = 2 \rightarrow dmin = 3 \cdot 2 = 6$, $t = \lfloor (6-1)/2 \rfloor = 2$: corrige hasta 2 y detecta hasta 5. Hamming en cada fila localiza la posición dentro de la fila; la paridad por columnas cruza la información.

Palabra del enunciado:

```
1 0 0 1
0 0 0 1
1 1 1 1
0 0 0
```

"¿Es válida?" — **Resolución del alumno:** no es válida: en la fila 2 ($0,0,0$) la suma es 0, la paridad debería ser 0 pero es 1; en la columna 2 ($0,0,1$) la suma es 1, la paridad debería ser 1 pero es 0.

[Nota] Con los datos tal como están transcriptos, **también falla la columna 3** ($0,0,1$, paridad 0): hay una fila y dos columnas con paridad errónea, que no es la huella de un solo error (un error simple hace fallar una fila y una columna que se cruzan). Además, si se cuenta el BPP de cada fila como parte de la palabra (Hamming extendido, $d_1 = 4$), el producto daría $dmin = 4 \cdot 2 = 8$; el enunciado habla de "palabra Hamming" y la alumna usó $d_1 = 3$. Otro alumno (Contreras, PARTE 1 bis) contesta $dmin = 4$: las dos respuestas difieren y no hay una resolución oficial en la carpeta.

Ejercicio 4 — Criptografía: sustitución afín

> El siguiente texto fue cifrado usando la sustitución afín, con clave (a, b) = (13, 9), n = 40. Texto cifrado: `U5K5CRUPKNLWNNWZAZY9K8EFQDURKJ886ERUPKIUKÑQ7`. Alfabeto: `ABCDEFGHIJKLMNÑOPQRSTUVWXYZ0123456789 /,` (A=0, B=1, …, Z=26, 0=27, …, 9=36, espacio=37, `/`=38, `,`=39). Encuentre: la clave inversa y el texto descifrado.
> Si $n = 1.000.003$, $a = 139$: $a^{-1} = ?$, $\phi(n) = ?$

Resolución del alumno. $M_i = (C_i - b) \cdot a^{-1} \bmod 40$. Como 40 no es primo, $\phi(40)$ se calcula con la fórmula de Euler: $40 = 2^3 \cdot 5$, $\phi(40) = 40 \cdot (1 - 1/2) \cdot (1 - 1/5) = 16$. $a^{-1} = a^{(\phi(40)-1)} \bmod 40 = 13^{15} \bmod 40 = 37$ (comprobación: $13 \cdot 37 = 481 = 12 \cdot 40 + 1$). La clave inversa es $M_i = (C_i - 9) \cdot 37 \bmod 40$.

Tabla de descifrado (c = índice del cifrado, $m = (c - 9) \cdot 37 \bmod 40$):

cifrado	U 5 K 5 C R U P K N L	W N N W Z A Z Y 9 K 8	E F Q D U R K J 8 8 6	E R U P K I U K Ñ Q 7
`c`	21 32 10 32 2 18 21 16 10 13 11	23 13 13 23 26 0 26 25 36 10 35	4 5 17 3 21 18 10 9 35 35 33	4 18 21 16 10 8 21 10 14 17 34
`m`	4 11 37 11 21 13 4 19 37 28 34	38 28 28 38 29 27 29 32 39 37 2	15 12 16 18 4 13 37 0 2 2 8	15 13 4 19 37 3 4 37 25 16 5
texto	`EL_LUNES_17`	`/11/2025,_C`	`OMPREN_ACCI`	`ONES_DE_YPF`

Texto descifrado: `EL LUNES 17/11/2025, COMPREN ACCIONES DE YPF` (el `_` es el espacio).

Segunda parte (la alumna dejó el cálculo sin terminar): `n = 1.000.003` es primo → `ϕ(n) = n − 1 = 1.000.002`; `a⁻¹ = 139^1.000.001 mod 1.000.003`. **[Completado]** `a⁻¹ = 107.914` (comprobación: `139·107.914 = 15.000.046 = 15·1.000.003 + 1`).

Ejercicio 5 — Entropía de orden 0

> ¿Qué algoritmo, de los vistos en clases, mantiene la entropía de orden 0 entre texto claro y texto cifrado, considerándolos como proceso de Markov de orden 0, pero altera las entropías de orden superior (1, 2, etc.)? Justifique su respuesta.

Resolución del alumno. "Sustitución monoalfabética": en un proceso de Markov de orden 0 la entropía depende solo de las probabilidades individuales, no del orden; un cifrado monoalfabético es una permutación de símbolos, no cambia la probabilidad de cada letra, solo se reasigna el símbolo → conserva `H₀`; la entropía de orden 1 considera transiciones y por eso importa la estructura del lenguaje.

[Corrección] La respuesta correcta es la **transposición**. La monoalfabética es una relabelación biyectiva: conserva las entropías de **todos** los órdenes (las probabilidades de cada par, terna, etc., solo cambian de nombre), así que no altera las de orden superior. La transposición reordena los mismos símbolos: conserva las frecuencias (`H₀` igual) y rompe las dependencias entre símbolos vecinos (`H₁, H₂, …` cambian).

Ejercicio 6 — Imágenes: filtro paso bajo

> Dé un ejemplo de filtro paso bajo, e indique qué efectos produce sobre una imagen.

Resolución del alumno. El filtro de mediana (reduce el ruido). Con la ventana 3×3

```
3  18 73
1  7  20
9  25 14
```

ordena los valores `1, 3, 7, 9, 14, 18, 20, 25, 73`, la mediana es `14` y reemplaza el valor central (7) por 14:

```
3  18 73
1  14 20
9  25 14
```

[Nota] En la transcripción de la alumna falta el `14` en la lista ordenada, pero la mediana es correcta. La mediana es un filtro de suavizado **no lineal**: elimina picos aislados (acá el 73) conservando mejor los bordes; un paso bajo lineal típico es la **media aritmética** (acá `170/9 ≈ 18,9 → 19`), que además de reducir el ruido emborrona los bordes.

PARTE 1 bis — Mismo parcial (17/11/2025, Tema 1), resolución de Contreras Matías

Archivo: `Parcial N2 de Teoria de la Informacion Tema 1 Contreras Matias.docx` (estaba en Descargas). El **enunciado es idéntico** al de la PARTE 1 (mismo Tema 1; solo escribe `g(x)` en vez de `G(x)`), así que no agrega ejercicios nuevos: lo que aporta es **una segunda resolución** para comparar.

Es una resolución incompleta: del Ejercicio 1 solo resuelve el inciso **a)**; los incisos **b)** (matrices), **c)**, **d)** y **e)** figuran solo como enunciado, sin resolución. Además el descifrado del afín se corta en `…2025` y la segunda parte del afín está mal (ver abajo).

Lo que contesta, inciso por inciso, y la verificación

Ejercicio	Resolución de Contreras	Verificación
1.a) Capacidad	"El peso de Hamming de `g(x)` es 6 (lo llama *cota inferior*), por lo tanto `dmin =`	**[Corrección]** el peso de `g(x)` es una cota **superior** de `dmin` (`g(x)` es una palabra del código, así que `dmin ≤ w(g)`), no inferior. Acá da

Ejercicio	Resolución de Contreras	Verificación
	`6`." `t = ⌊5/2⌋ = 2`, detecta 5.	igual a `dmin` por coincidencia; el método correcto es el mínimo peso de las 15 palabras no nulas (ver PARTE 1, 1.a). Resultado `6 / 2 / 5` correcto; falta decir que como corrector corrige 2 y detecta 3 a la vez.
2) ISBN-13 `978-1-80056-662-3`	Tabla de productos `9, 21, 8, 3, 8, 0, 0, 15, 6, 18, 6, 6, 3` = **103**; "`104 mod 10 = 3`"; no es válido. Corrige el dígito de control a `0`: `978-1-80056-662-0` (suma `100`, `100 mod 10 = 0`).	Correcto. **[Nota]** "104" es un error de tipeo (la suma es 103, y `103 mod 10 = 3` es lo que usa después). Igual que en la PARTE 1, la corrección solo vale suponiendo que el error está en el dígito de control; no lo aclara.
3) Código rectangular	"Para un código Hamming rectangular `m×n`, `dmin = 4`: detecta 2 y corrige 1." Chequea las paridades: fila 0 no falla, **fila 1 falla**, fila 2 no falla; columna 0 no falla, **columna 1 falla**, **columna 2 falla** → "el código no es válido".	Las paridades de la palabra están bien (**[Nota]** coinciden con las de la PARTE 1: fila 2 y columnas 2 y 3 fallan, es decir, 1 fila + 2 columnas, que no es la huella de un error simple). **El `dmin` difiere del de la PARTE 1** (4 contra 6): ver la comparación de abajo.
4) Afín `(13, 9)`, `n = 40`	`40 = 2³·5`, `ϕ(40) = 40·(1 − 1/2)(1 − 1/5) = 16`; CRR(40) = `{1, 3, 7, 9, 11, 13, 17, 19, 21, 23, 27, 29, 31, 33, 37, 39}`; `a⁻¹ = 13¹⁵ mod 40 = 37`. Tabla de descifrado hasta `EL LUNES 17/11/2025`.	`ϕ(40)`, el CRR, `a⁻¹ = 37` y los 19 primeros símbolos son correctos. **El texto está sin terminar:** faltan `, COMPREN ACCIONES DE YPF` (los 25 símbolos restantes).
4) Afín, segunda parte `n = 1.000.003`, `a = 139`	`n` es primo ⟶ `ϕ(n) = 1.000.002`. Escribe "`137⁻¹ = 137^1.000.001 mod 1.000.003 → 137^1.000.002 mod 1.000.003 = 167369`".	**[Corrección]** tres errores: (i) `a` es **139**, no 137; (ii) el exponente es `ϕ(n) − 1 = 1.000.001`, no `1.000.002` (con `ϕ(n)` el resultado sería 1, por Fermat); (iii) **167369 no es el inverso** ni de 137 (da 824.820) ni de 139. El correcto es `139⁻¹ = 107.914` (verificado: `139·107.914 = 15.000.046 = 15·1.000.003 + 1`).
5) Entropía de orden 0	**La transposición**: reordena las letras sin cambiar cuántas veces aparece cada una y, al cambiar el orden, rompe las dependencias entre símbolos (las de orden superior cambian).	**Correcto**, y coincide con la corrección de la PARTE 1 (Ej. 5). Es la segunda resolución que llega a esa respuesta.
6) Filtro paso bajo	Filtro de **media**. Ventana `12 8 15 / 6 22 10 / 18 14 9`: suma `114`, `114/9 = 12,67 ≈ 13`; el centro `22` pasa a `13`: `12 8 15 / 6 13 10 / 18 14 9`. Efecto: imagen más borrosa y menos ruido.	Correcto (`12+8+15+6+22+10+18+14+9 = 114`). **[Nota]** Es el paso bajo lineal "de manual"; complementa el ejemplo de la mediana de la PARTE 1 (Ej. 6).

Lo que conviene saber de este parcial

1. **El `dmin` del código rectangular tiene dos respuestas distintas.** La PARTE 1 contesta **6** (`d₁·d₂ = 3·2`: filas Hamming × columnas paridad simple); Contreras contesta **4** (corrige 1 y detecta 2). Ninguna de las dos es una resolución oficial. Los números salen de reglas distintas de la cátedra: el código **producto** da `d₁·d₂`; el rectangular de paridades **sin bit esquina** tiene `dmin = 3` y **con bit esquina** `dmin = 4` (Práctico de Control de Error, ejercicio 4). **[Nota]** En la figura del enunciado la celda de la esquina está vacía (no hay bit esquina), y las filas son "palabra Hamming" + `BPP`. Como no hay forma de saber cuál toma el docente, en el examen conviene **escribir qué regla se aplica y por qué** (producto de códigos, o rectangular con/sin esquina) y comprobar que la capacidad sale consistente con ese `dmin`.
2. **La transposición como respuesta a "conserva `H₀` y altera las superiores"** queda confirmada por dos resoluciones (la monoalfabética de la PARTE 1 era un error).
3. **El peso de `g(x)` es solo una cota superior de `dmin`.** Sirve para chequear (`dmin ≤ w(g)`), no para afirmar `dmin`.
4. **Cuidado al copiar los datos del enunciado** (137 en vez de 139) y con el exponente de Euler (`ϕ(n) − 1`): el resultado `167369` mostró que con un par de errores el número final no tiene nada que ver con el inverso.
5. **Las dos resoluciones coinciden** en `103 mod 10 = 3`, el dígito de control `0`, el afín `37` y el texto descifrado: esos resultados están doblemente confirmados (y verificados por programa).

PARTE 2 — Parcial 15/12/2020 (`Johana Arce Parcial N°2 … CORREGIDO-APROBADO.pdf`, alumna Arce)

Puntaje total: 7,2. Desglose según los comentarios del docente: Control de error 3,2/3,5; ISBN 2/3; Criptografía 2/2; Imágenes 0/1,5 ("FALTA IMÁGENES").

Enunciado

> **1) CONTROL DE ERROR (3,5P):** Sea el siguiente polinomio $h(X) = X^3 + X^2 + X + 1$ de un código (n, k, dmin) (12, ?, ?). **A)** Encontrar la cantidad de dígitos de información k que constituyen la palabra válida. **B)** Encontrar el valor de la distancia mínima indicando las cualidades del código funcionando como corrector y como detector. **C)** Encuentre el polinomio Generador $g(X)$. **D)** Encuentre la matriz generador G y chequeadora H. **E)** Explique qué sucede, en la instancia decodificadora, ante la presencia de un error triple!! Y qué sucedería si se llegara a comparar el síndrome con las filas de H^T?
>
> **2) Dado el siguiente ISBN (3P) A)** encontrar el valor de dígito faltante (?) según la forma de codificación con solamente 10 dígitos (versión anterior del código) y no con 13 (versión actual del código) $3-540-?3218-9$. **B)** Qué características tiene el código y cuál es su distancia mínima. **C)** Cuál es la justificación por la cual se usa la base prima en toda instancia de codificación para el control de error?
>
> **Criptografía (2p)**
> **3)** Dado el siguiente criptograma $SGHPKQ\$SGT$, se sabe que fue cifrado con una sustitución afín, con clave (a,b) = (3,11), n = 29, usando el siguiente alfabeto A=0, B=1, …, Ñ=14, …, Z=26, $=27, #=28. Se pide: encontrar los inversos de a y de b, y decodificar el criptograma.
>
> **Imágenes (1,5p)**
> **4) a)** Cuál es la diferencia entre una imagen Vectorial y una imagen bitmapeada? **b)** Dada la siguiente secuencia de valores de color de una imagen: $129, 129, 129, 129, 129, 194$, codificarla usando la codificación RLE de PCX.

Resolución del alumno + correcciones del docente

1) Control de error — código (12, 3), ver tabla 0.2

- **A)** "Se deduce que $k = 3$, ya que el polinomio h(x) por definición es de grado k." ✓
- **B)** "La distancia mínima, de acuerdo a la cantidad de términos del polinomio, es $dmin = 6$; por lo tanto el código tiene capacidad de detectar hasta 5 errores y corregir 2."
  [Corrección] del docente: *"COMO CORRECTOR, CORRIGE HASTA DOS Y DETECTA 3!!"* ($dmin = 6 \geq t + s + 1 = 2 + 3 + 1$: usado como corrector corrige 2 y a la vez detecta 3 más; la "detección hasta 5" es solo si se usa como detector puro). Verificado: los pesos de las 7 palabras no nulas son 6 y 12 → $dmin = 6$.
- **C)** $g(x) = (x^{12} + 1)/h(x)$: divide $x^{12} + 1$ por $x^3 + x^2 + x + 1$ y obtiene $g(x) = x^9 + x^8 + x^5 + x^4 + x + 1$ (resto 0). ✓
- **D)** Arma la G sistemática $[P \mid I_3]$ con las filas $x^{n-k+i} \bmod g$ (para $i = 0, 1, 2$ calcula x^9, x^{10}, x^{11} módulo $g(x)$):

  ```
  G =  110011001 100
       101010101 010
       100110011 001        (P | I3)
  ```

  y $H = [I_9 \mid P^T]$ (9×12).
  [Nota] La G manuscrita coincide con la calculada. La H manuscrita es de 9 filas con una identidad de 9×9 y 3 columnas de P^T; con la letra del escaneo no pude confirmar cada fila. La H correcta es la de la tabla 0.2 ($G \cdot H^T = 0$ verificado).
- **E)** Respuesta de la alumna: al dividir el mensaje erróneo por $g(x)$ el resto no es 0 → se detecta que no es palabra válida; ese resto es el síndrome, que al compararlo con las filas de H^T (o las columnas de H) indica el error: error simple coincide con una sola fila, doble con el XOR de 2 filas, etc. Ante un error triple el síndrome coincide con el XOR de 3 filas, pero como el código solo corrige hasta 2 no puede corregirlo, solo detectarlo.
  [Corrección] del docente (Pje 3,2/3,5): *"TODO BIEN!! LO ÚNICO ES QUE ANTE LA PRESENCIA DE UN ERROR TRIPLE, SI SE HICIERA LA COMPARACIÓN DEL SÍNDROME CON LAS FILAS DE H TRASP HABRÍAN DOS GRUPOS DE TRES FILAS CUYA COMBINACIÓN COINCIDIRÍA CON EL VALOR DEL SÍNDROME. POR ELLO NO PUEDE APROXIMAR A NINGUNA DE ESAS PALABRAS Y PIDE RETRANSMISIÓN Y NO GENERA UN TERCER ERROR. EN REALIDAD COMO SE SABE UN CORRECTOR DOBLE DIRECTAMENTE SI EL SÍNDROME NO COINCIDE NI CON UNA FILA O COMBINACIÓN DE DOS FILAS, PIDE RETRANSMISIÓN!!"*
  [Completado] Verificación por cómputo: de los $C(12,3) = 220$ errores triples, **ninguno** tiene un síndrome igual al de un error simple o doble (por eso siempre se detecta y nunca se "corrige mal" hacia una palabra a distancia ≤ 2); y de los 163 síndromes distintos que producen, **55 se explican con 2 (o 4) combinaciones distintas de tres filas** (p. ej. posiciones $\{0,1,4\}$ y $\{5,8,9\}$ dan el mismo síndrome), de modo que el decodificador no sabría a cuál palabra corregir → pide retransmisión.

2) ISBN-10 $3-540-?3218-9$

- **A)** Pesos $10, 9, \ldots, 1$; la alumna suma 113 y concluye que $? = 6$ ($11 \cdot 13 = 143$, $143 - 113 = 30$, $6 \cdot 6 = 30$).
  [Corrección] del docente: *"SE EQUIVOCÓ EN LA SUMA PARA LA OBTENCIÓN DEL DÍGITO FALTANTE ?"* — Pje 2/3.
  Cálculo correcto: $3 \cdot 10 + 5 \cdot 9 + 4 \cdot 8 + 0 \cdot 7 + 6x + 3 \cdot 5 + 2 \cdot 4 + 1 \cdot 3 + 8 \cdot 2 + 9 \cdot 1 = 158 + 6x$; $158 \bmod 11 = 4$ → $6x \equiv -4 \equiv 7 \pmod{11}$ → $x = 7 \cdot 6^{-1} = 7 \cdot 2 = 14 \equiv 3$. $3-540-33218-9$ (comprobación: $158 + 18 = 176 = 16 \cdot 11$).
- **B)** "Es un código pesado: a cada dígito, según su posición, se lo multiplica por un peso. Su $dmin$ es 2" → detecta 1 error, no corrige. ✓
- **C)** "Se usa base prima porque permite una codificación unívoca." **[Corrección] del docente:** *"DECODIFICACIÓN UNÍVOCA!!"* (ver 0.4: todo peso tiene inverso módulo un primo → el dígito faltante es único).

3) Afín `(a, b) = (3, 11)`, `n = 29`, alfabeto `A…Z` con `Ñ = 14`, `$ = 27`, `# = 28`

`a⁻¹`: `3·x ≡ 1 (mod 29)` → `x = 10` (`3·10 = 30 ≡ 1`). El inverso de `b`: `10·11 + b' ≡ 0 (mod 29)` → `b' = −110 mod 29 = 6` (es la `b'` de 0.5). Decodificación `D(m) = a⁻¹·c + b' = 10·c + 6 mod 29`:

cifrado	S	G	H	P	K	Q	$	S	G	T
`c`	19	6	7	16	10	17	27	19	6	20
`10c + 6 mod 29`	22	8	18	21	19	2	15	22	8	3
texto	V	I	R	U	S	C	O	V	I	D

Texto descifrado: `VIRUS COVID` (`$` = espacio). **Docente:** *"BIEN 2/2"*.

4) Imágenes — el escaneo no trae resolución (**Docente:** *"FALTA IMÁGENES 0/1.5"*)

[Completado]

- **a)** Una imagen **vectorial** se describe con figuras matemáticas (puntos, rectas, curvas, rellenos): escala sin pérdida de calidad y ocupa poco; una **bitmap** (mapa de bits) es una matriz de píxeles: ocupa más espacio, depende de la resolución y se pixela al ampliarla.
- **b)** RLE de PCX de `129, 129, 129, 129, 129, 194`: la racha de cinco 129 → byte `0xC0 + 5 = 0xC5 (197)` seguido de `0x81 (129)`; el `194` (`≥ 192`) aparece una vez y necesita el byte de escape `0xC1 (193)` antes del valor `0xC2 (194)`. Resultado: `197, 129, 193, 194` (hex `C5 81 C1 C2`).

PARTE 3 — Parcial 29/11/2010 (`PARCIAL2TI.doc.pdf`) — enunciado + [Completado]

> **PARCIAL N° 2 DE COMUNICACIONES II (TEORÍA DE INFORMACIÓN)** — FECHA: 29/11/2010
>
> **Ejercicio 1) Criptografía:** El siguiente criptograma `LFXOJKLWEOPKYDJQPJUCADCG` fue encriptado usando secuencialmente, primero un método de Vigenère con clave `RACING`, y posteriormente una clave de permutación `(6,5,4,3,2,1)`. **A)** Encuentre el mensaje original **B)** Calcule los valores de entropía (como modelo independiente) que adquiere la fuente en cada etapa de encriptación **C)** Especifique qué niveles de entropía (de orden 0 —independiente—, o de orden 1 —markov de orden 1—, o de orden 2 etc...) modifica cada una de las estrategias utilizadas. **D)** Calcule la redundancia de la fuente original. (El universo de símbolos considerado es: ABCDEFGHIJKLMNOPQRSTUVWXYZ, A=1 … Z=26).
>
> **Ejercicio 2) Control de error:** Sea el siguiente polinomio `h(x) = x³ + x² + x + 1` de un código (12, K, dmin). **A)** Encuentre la capacidad del código. **B)** Especifique, como combinación de qué bits de información se obtiene cada uno de los chequeadores. **C)** Decodifique el stream de bits recibido `110011001110` en caso de haber algún error corrija y obtenga la palabra válida (las mayores potencias de x están a la derecha). **D)** Dibuje el circuito de `h(x)`.
>
> **Ejercicio 3) Teóricas: A)** Ubique los siguientes archivos en orden decreciente de entropía relativa (justifique) `*.exe; *.txt; *.rar`. **B)** Si se entrelazan 7 códigos del tratado en el punto dos, qué nuevas capacidades adquiere el código. **C)** Marque las diferencias entre criptografía simétrica y asimétrica. **D)** Cuál de las tres fuentes, del ejercicio 1 (la encriptada final, la encriptada intermedia o la original) conviene comprimir (justifique).

Ejercicio 1 — [Completado]

A) Mensaje original. Cifrado = Vigenère primero, permutación después → se deshace la permutación y luego Vigenère.

1. Permutación `(6,5,4,3,2,1)` sobre bloques de 6: `C_j = M_{k_j}` → cada bloque se lee al revés: `LFXOJK | LWEOPK | YDJQPJ | UCADCG` → `KJOXFL | KPOEWL | JPQJDY | GCDACU` = `KJOXFLKPOEWLJPQJDYGCDACU` (mensaje intermedio, ya "Vigenerizado").
2. Vigenère con `RACING` (`R=18, A=1, C=3, I=9, N=14, G=7`), `M = (C − K) mod 26` escrito en `1…26` (0 ≡ 26):

	bloque 1	bloque 2	bloque 3	bloque 4
cifrado intermedio	K J O X F L	K P O E W L	J P Q J D Y	G C D A C U
`C` (valores, A=1)	11 10 15 24 6 12	11 16 15 5 23 12	10 16 17 10 4 25	7 3 4 1 3 21
`K` (clave `RACING` repetida)	18 1 3 9 14 7	18 1 3 9 14 7	18 1 3 9 14 7	18 1 3 9 14 7
`M = (C − K) mod 26` (0 ≡ 26)	19 9 12 15 18 5	19 15 12 22 9 5	18 15 14 1 16 18	15 2 1 18 15 14
mensaje	S I L O R E	S O L V I E	R O N A P R	O B A R O N

Mensaje original: `SILORESOLVIERONAPROBARON` → "SI LO RESOLVIERON APROBARON".

[Nota] La numeración `A = 1` importa: con `A = 0` el mismo cálculo da `TJMPSFTPMWJFSPOBQSPCBSPO` (todo corrido una letra).

B) Entropía de orden 0 (modelo independiente) en cada etapa ($H = \Sigma\ p_i \cdot \log_2(1/p_i)$, 24 símbolos):

Etapa	Texto	Símbolos distintos	Frecuencias	H_0 (bits/símbolo)
Original	`SILORESOLVIERONAPROBARON`	11	O 5, R 4, A/E/I/L/N/S 2 c/u, B/P/V 1 c/u	**3,2679**
Tras Vigenère (antes de permutar)	`KJOXFLKPOEWLJPQJDYGCDACU`	16	J 3; C/D/K/L/O/P 2 c/u; 9 letras con 1	**3,8868**
Tras la permutación (final)	`LFXOJKLWEOPKYDJQPJUCADCG`	16	(las mismas que la etapa anterior)	**3,8868**

(Ejemplo del cálculo del original: $5/24 \cdot \log_2(24/5) + 4/24 \cdot \log_2 6 + 6 \cdot (2/24) \cdot \log_2 12 + 3 \cdot (1/24) \cdot \log_2 24 = 3{,}2679$.)

C) Qué niveles modifica cada estrategia. La **permutación (transposición)** no modifica H_0 (3,8868 antes y después: los mismos símbolos con las mismas frecuencias) pero sí las de orden ≥ 1 (cambia qué símbolo sigue a cuál). **Vigenère** modifica H_0 (3,2679 → 3,8868, las frecuencias se aplanan porque un mismo símbolo se cifra distinto según su posición) y también las de orden superior.

D) Redundancia de la fuente original. $H_{max} = \log_2 26 = 4{,}7004$; $R = 1 - H/H_{max} = 1 - 3{,}2679/4{,}7004 = 0{,}3048$ → **30,5 %**.

Ejercicio 2 — [Completado]

Es el código **(12, 3)** de la tabla 0.2: $k = 3$, $g(x) = (x^{12} + 1)/h(x) = 1 + x + x^4 + x^5 + x^8 + x^9$.

A) Capacidad. Palabras no nulas con peso 6 o 12 → $d_{min} = 6$. Como detector detecta hasta 5; como corrector corrige 2 (y detecta 3 más a la vez). Redundancia $N/K = 12/3 = 4$.

B) Chequeadores (información `I1 I2 I3` en las mayores potencias):
$C1 = I1 \oplus I2 \oplus I3$, $C2 = I1$, $C3 = I2$, $C4 = I3$, $C5 = I1 \oplus I2 \oplus I3$, $C6 = I1$, $C7 = I2$, $C8 = I3$, $C9 = I1 \oplus I2 \oplus I3$ (matrices G y H en 0.2).

C) Decodificar `110011001110` (mayores potencias a la derecha): $R(x) = 1 + x + x^4 + x^5 + x^8 + x^9 + x^{10} = g(x) + x^{10}$. Resto de $R \div g$ = $x^{10} \bmod g = 1 + x^2 + x^4 + x^6 + x^8$ (`101010101`) ≠ 0 → **no es válida**. Ese síndrome es el de un **error simple en la posición** x^{10} (segundo bit de información, `I2`). Se invierte ese bit: `110011001110` → `110011001100` , que es $g(x)$ (resto 0) → **palabra válida** `110011001100` , información `I1 I2 I3 = 1 0 0` .

D) Circuito de $h(x) = 1 + x + x^2 + x^3$ **.** Registro de desplazamiento de **3 celdas** (grado de h), con realimentación por XOR de las celdas cuyos coeficientes valen 1 (acá las tres):



[Nota] Lo que se evalúa es que las conexiones coincidan con los coeficientes del polinomio; el sentido de las conexiones depende de si el diagrama es de Fibonacci (criptografía) o de Galois (control de error): conviene confirmarlo con la presentación de la cátedra.

Ejercicio 3 — [Completado]

A) Orden decreciente de entropía relativa (H/H_{max} **):** `*.rar` **>** `*.exe` **>** `*.txt` **.** Un archivo comprimido (`.rar`) tiene los bytes casi equiprobables (la compresión elimina la redundancia) → entropía relativa casi 1; el ejecutable (`.exe`) tiene una mezcla de código máquina y datos con estructura moderada; el texto plano (`.txt`) tiene la distribución más desigual (muchas apariciones de pocas letras y espacios) → la entropía relativa más baja.

B) Si se entrelazan 7 códigos del (12, 3): adquiere capacidad de corrección de **ráfagas**: una ráfaga de hasta $7 \cdot t = 7 \cdot 2 = 14$ bits consecutivos (los bits consecutivos caen en 7 palabras distintas, cada una con a lo sumo 2 errores, que sabe corregir); la capacidad de errores aleatorios por palabra sigue siendo $t = 2$.

C) Simétrica vs. asimétrica (ver tabla en la PARTE 6).

D) ¿Cuál conviene comprimir? La **original**: es la de menor entropía (3,2679) y mayor redundancia (30,5 %); las dos encriptadas tienen la entropía aplanada (3,8868, redundancia 17,3 %) porque el cifrado destruye las regularidades estadísticas que el compresor necesita.

Por eso se **comprime antes de cifrar**.

PARTE 4 — Parcial 21/11/2012 (`Parcial 22012-TI.doc.pdf`) — enunciados + [Completado]

4.1 Tema 1

> **CONTROL DE ERROR:** Sea el código (15,K?, dmin?) generado por el siguiente polinomio generador $h(X) = X^4 + X + 1$. **a)** Encuentre el polinomio generador $g(X)$ **b)** Determine las capacidades del código si funciona como corrector y/o detector, encontrando la distancia mínima **c)** Especifique como combinación lineal de qué dígitos de información?, se obtiene cada chequeador. (mayores potencia de G a la derecha) **d)** Determine si el polinomio $R(x) = X^8 + X^4 + X^2$ es válido o no. Si no lo es corríjalo. **e)** Obtenga la capacidad de corrección a nivel de ráfaga si se entrelazan 20 códigos generados por $h(x)$.
>
> **CRIPTOGRAFÍA:** Dado el siguiente criptograma, `GAUQ YBXM OÑSK`, descifrarlo, sabiendo que fue cifrado utilizando una transposición de una matriz de 3x4, en la que se pusieron las letras por fila y se leyeron por columna según el siguiente orden $(4,1,3,2)$, el resultado fue cifrado utilizando la técnica de Vigenère con clave `CANCUN`. **B)** Los métodos antes vistos modifican la entropía de orden cero? **C)** En la fuente cifrada, y cada paso de descifrado calcule la entropía de orden cero para comprobar lo manifestado por usted en el punto anterior.
>
> **IMÁGENES:** Dada la siguiente imagen, utilizando las máscaras pertinentes, determinar a qué tipo de recta pertenece el punto central (horizontal, vertical, a 45°, a 135°): `19 3 21 / 4 19 11 / 16 16 19`
>
> **TEÓRICAS:** 2) En un buen sistema criptográfico cuánto debe valer la Información Mutua I(A,B) Asumiendo que A es el mensaje original y B es el mensaje cifrado? Justifique. 3) Escriba las propiedades del polinomio generador de un código cíclico.

Resolución — [Completado]

Control de error. Es el código **(15, 4)** de la tabla 0.2.

- **a)** $k = \text{grado de } h = 4$; $g(x) = (x^{15} + 1)/(x^4 + x + 1) = 1 + x + x^2 + x^3 + x^5 + x^7 + x^8 + x^{11}$ (111101011001, grado 11).
- **b)** Las 15 palabras no nulas pesan todas 8 (es el código símplex) → $\mathbf{dmin = 8}$: como detector detecta 7; como corrector corrige 3 (y detecta 4 más a la vez: $3 + 4 + 1 = 8$).
- **c)** Chequeadores: $C1 = I1 \oplus I4$, $C2 = I1 \oplus I2 \oplus I4$, $C3 = I1 \oplus I2 \oplus I3 \oplus I4$, $C4 = I1 \oplus I2 \oplus I3$, $C5 = I2 \oplus I3 \oplus I4$, $C6 = I1 \oplus I3$, $C7 = I2 \oplus I4$, $C8 = I1 \oplus I3 \oplus I4$, $C9 = I1 \oplus I2$, $C10 = I2 \oplus I3$, $C11 = I3 \oplus I4$ (matrices G, H en 0.2).
- **d)** $R(x) = x^2 + x^4 + x^8$ (grado 8 < 11, ya está reducido): resto = $R \neq 0$ → **no es válido**. El único patrón de peso ≤ 3 con ese síndrome es $e = x^2 + x^4 + x^8$ (error triple, justo en la capacidad $t = 3$) → corregida $R + e = 0$ = **palabra nula** (válida).
- **e)** Con 20 códigos entrelazados: ráfaga corregible $20 \cdot t = 20 \cdot 3 =$ **60** bits.

Criptografía. Cifrado: transposición 3×4, clave $(4,1,3,2)$, y *después* Vigenère `CANCUN` → se deshace Vigenère y luego la transposición.

1. Vigenère inverso (27 letras, $A = 0$, $M = C - K$, clave `CANCUN`): `G−C=E, A−A=A, U−N=I, Q−C=O, Y−U=E, B−N=O, X−C=V, M−A=M, O−N=C, Ñ−C=M, S−U=Y, K−N=X` → **`EAIOEOVMCMYX`**.
2. Transposición (convención "la columna j se lee en la posición k_j"): matriz de 3 filas × 4 columnas escrita por filas; la columna 1 se leyó 4.ª, la 2 en 1.ª, la 3 en 3.ª y la 4 en 2.ª. Se separa `EAI | OEO | VMC | MYX` (columnas 2, 4, 3, 1) y se rearma por filas:

```
M E V O
Y A M E
X I C O
```

Mensaje original: `MEVOYAMEXICO` → "ME VOY A MÉXICO". (Verificación: recifrar da `GAUQYBXMOÑSK`.)

B) La **transposición no modifica la entropía de orden 0** (mismos símbolos y frecuencias); **Vigenère sí** (polialfabético con una clave de varias letras distintas).

C) Entropía de orden 0 en cada paso ($Hmax = \log_2 27 = 4{,}7549$; en orden de descifrado):

Paso	Texto	Símbolos distintos	H_0
Cifrado (fuente recibida)	`GAUQYBXMOÑSK`	12 (todos distintos, equiprobables)	**3,5850** (= $\log_2 12$)
Tras deshacer Vigenère	`EAIOEOVMCMYX`	9 (E, M, O ×2; el resto ×1)	**3,0850**
Tras deshacer la transposición (claro)	`MEVOYAMEXICO`	9 (iguales frecuencias)	**3,0850**

Se comprueba lo anterior: Vigenère cambia H_o (3,5850 ↔ 3,0850) y la transposición no (3,0850 en ambos lados).

Imágenes. Aplicadas las cuatro máscaras (0.7) a la ventana `19 3 21 / 4 19 11 / 16 16 19`:
Máscara	horizontal	vertical	45°	135°
Respuesta	−26	−14	40	**43**

La mayor respuesta en valor absoluto es la de 135° → el punto central pertenece a una recta a **135°**. **[Nota]** La diferencia con 45° (40 contra 43) es chica, y la numeración de los ángulos depende de la convención de las máscaras de la cátedra.

Teóricas. 2) En un buen sistema criptográfico `I(A,B)` debe ser **mínima**, idealmente **0** (secreto perfecto de Shannon: el criptograma no da información sobre el mensaje); en la práctica nunca es exactamente 0. 3) Propiedades de `g(x)` de un cíclico: **(i)** es de grado `n − k`; **(ii)** es factor de $x^n + 1$; **(iii)** es el polinomio de menor grado del código (y primo, según la cátedra); **(iv)** toda palabra válida `V(x)` es múltiplo de `g(x)` y todo múltiplo de `g(x)` (de grado < `n`) es palabra válida; **(v)** toda rotación de una palabra válida es otra palabra válida; además tiene coeficiente 1 en el término independiente y en el de mayor grado.

4.2 Tema 2

> **CONTROL DE ERROR:** Sea el código (14,K?, dmin?) generado por el siguiente polinomio generador $h(X) = X^4 + X^2 + X + 1$. **a)** Encuentre el polinomio generador `g(X)` **b)** Determine las capacidades del código si funciona como corrector y/o detector, encontrando la distancia mínima **c)** Especifique como combinación lineal de qué dígitos de información?, se obtiene cada chequeador. (mayores potencia de G a la derecha) **d)** Determine si el polinomio $R(x) = X^8 + X^4$ es válido o no. Si no lo es corríjalo. **e)** Obtenga la capacidad de corrección a nivel de ráfaga si se entrelazan 20 códigos generados por `h(x)`.
>
> **CRIPTOGRAFÍA:** Dado el siguiente criptograma, `BOEU MIHJ FDFX DVDT`, descifrarlo, sabiendo que fue cifrado utilizando la técnica de Vigenère con clave `AZULES`, y al resultado se lo dispuso por filas en una matriz de 4x4, y luego leído por columnas según el siguiente orden `(2,4,1,3)`. Cuál de las dos alternativas usadas garantiza mayor seguridad o capacidad de encriptación? Justifique.
>
> **IMÁGENES:** Dada la siguiente imagen, utilizando la máscara pertinente, suavizar el punto central (de las dos ventanas), utilizando una máscara de media aritmética: `160 161 163 180 / 200 11 21 190 / 199 161 165 191`
>
> **TEÓRICAS:** 1) justifique la necesidad de la base prima. 2) Calcule el dígito faltante en el siguiente ISBN: `234323?323` 3) Llene el siguiente cuadro con información asociada a campos de Galois y códigos Reed-Solomon:

Tamaño del campo de Galois GF(q)	Grado del polinomio generador del campo de Galois GF(q)	Longitud de palabra válida en bytes (n)	Cantidad de bits por byte	Cantidad de bytes chequeadores	Cantidad de bytes de información	Capacidad de corrección del código (t)	Si entrelazo 7 códigos de este tipo puedo corregir una ráfaga de cuántos bytes?
2048						100 bytes	
	9				461		
		127		10			

Resolución — [Completado]

Control de error. Es el código **(14, 4)** de la tabla 0.2 (el mismo de 2013 T1 y 2025).

- **a)** $g(x) = (x^{14} + 1)/(x^4 + x^2 + x + 1) = 1 + x + x^3 + x^7 + x^8 + x^{10}$ (`11010001101`).
- **b)** `dmin = 6` (pesos 6, 8 y 14): corrige 2 (y detecta 3 más) / detecta 5 como detector.
- **c)** `C1 = I1⊕I3⊕I4`, `C2 = I1⊕I2⊕I3`, `C3 = I2⊕I3⊕I4`, `C4 = I1`, `C5 = I2`, `C6 = I3`, `C7 = I4`, `C8 = I1⊕I3⊕I4`, `C9 = I1⊕I2⊕I3`, `C10 = I2⊕I3⊕I4`.
- **d)** $R(x) = x^4 + x^8$ (grado 8 < 10): resto = `R` ≠ 0 → **no es válido**. No es un error simple pero sí un **error doble en las posiciones** x^4 **y** x^8 (único patrón de peso ≤ 2 con ese síndrome) → corregida = $R + x^4 + x^8 = 0$ = **palabra nula** (válida).
- **e)** $20 \cdot t = 20 \cdot 2 =$ **40** bits.

Criptografía. Cifrado = Vigenère `AZULES` primero, matriz `4×4` (por filas, leída por columnas en el orden `(2,4,1,3)`) después → se deshace la transposición y luego Vigenère.

1. Transposición inversa (convención "la columna `j` se lee en la posición k_j"): la columna 1 se leyó 2.ª, la 2 en 4.ª, la 3 en 1.ª y la 4 en 3.ª. El cifrado se parte en 4 bloques de 4 letras: `BOEU` (columna 3), `MIHJ` (columna 1), `FDFX` (columna 4), `DVDT` (columna 2). Se escribe la matriz por columnas y se lee por filas:

```
M D B F
I V O D
```

```
H D E F
J T U X
```

→ `MDBFIVODHDEFJTUX` (el texto ya "Vigenerizado").
2. Vigenère inverso con `AZULES` (27 letras, `A = 0`): `MEHUEDOENSANJUAN` → "ME HUEDO EN SAN JUAN".

[Nota] Probablemente el mensaje buscado es **"ME QUEDO EN SAN JUAN"**: si se cifra `MEQUEDOENSANJUAN` con este mismo procedimiento se obtiene `LOEU MIHJ FDFX DVDT`, es decir, **idéntico salvo la primera letra** (el enunciado trae `B` donde saldría `L`). El resultado del algoritmo sobre el criptograma tal como está escrito es `MEHUEDOENSANJUAN`.

¿Cuál de las dos alternativas da más seguridad? Vigenère (sustitución polialfabética = **confusión**): modifica la entropía de orden 0 (3,1556 del claro → 3,5778 tras Vigenère) y aplana las frecuencias; la **transposición** (**difusión**) deja `H₀` intacta (3,5778 antes y después de la matriz) y solo reordena, así que un análisis de frecuencias de letras individuales la delata. Por separado, Vigenère da más seguridad; combinadas se refuerzan (confusión + difusión).

Imágenes. Media aritmética 3×3 sobre las dos ventanas de la imagen 3×4:

- Ventana de las columnas 1–3 (centro `11`): suma `160+161+163+200+11+21+199+161+165 = 1241` → `1241/9 = 137,9 ≈` **138**.
- Ventana de las columnas 2–4 (centro `21`): suma `161+163+180+11+21+190+161+165+191 = 1243` → `1243/9 = 138,1 ≈` **138**.

Los dos centros (11 y 21, que eran picos de ruido frente a sus vecinos) pasan a 138.

Teóricas.

1. **Base prima:** ver 0.4 — en $\mathbb{Z}_p$ todo no nulo tiene inverso, el dígito faltante es único y la decodificación es unívoca (sin ambigüedades).
2. **ISBN `234323?323`:** pesos `10…1`: `2·10 + 3·9 + 4·8 + 3·7 + 2·6 + 3·5 + 4x + 3·3 + 2·2 + 3·1 = 143 + 4x`; `143 = 13·11 ≡ 0` → `4x ≡ 0 (mod 11)` → `x = 0` (`2343230323`, suma `143`).
3. **Cuadro de Galois / Reed–Solomon** (`q = 2^m`, `n = q − 1`, `n − k = 2t`, ráfaga con 7 códigos = `7·t`):

`q`	grado del polinomio del campo (`m`)	`n` (bytes)	bits por byte	bytes chequeadores (`2t`)	bytes de información (`k`)	`t`	ráfaga con 7 códigos
2048	**11**	**2047**	**11**	**200**	**1847**	100 bytes	**700 bytes**
512	9	**511**	**9**	**50**	461	**25**	**175 bytes**
128	**7**	127	**7**	10	**117**	**5**	**35 bytes**

(En negrita lo que se completó; el resto es dato del enunciado.)

PARTE 5 — Parcial 12/11/2013 (`Parcial 2 - TI 2013.doc.pdf`) — enunciados + [Completado]

5.1 Tema 1

> **a) CRIPTOGRAFÍA: A1)** Dado el siguiente texto cifrado: `Q V V Ñ M W H F V Y M Q S V K Ñ T K A G`, se sabe que fue cifrado con cifrador compuesto, utilizando los algoritmos: Vigenère con clave `K1 = (K,E,R,B,E,R,O,S)` y luego una transposición, utilizando una matriz de 4x5, con clave `K2 = (2,5,3,1,4)` (el texto en matriz fue dispuesto por filas). Encuentre las claves para descifrar el texto y el texto en claro. **A2)** Defina confusión y difusión. Identifique en el punto anterior si alguno de los algoritmos introdujo confusión, difusión o ambos. **A3)** Para cada algoritmo, indique si se modificaron las entropías de orden cero (Independiente) y/o de orden 1. Bajo este concepto cuál de los dos algoritmos es mejor? Justifique su respuesta.
>
> **B) CONTROL DE ERROR:** Sea el código (14,K?, dmin?) generado por el siguiente polinomio generador `h(X) = X⁴ + X² + X + 1`. **a)** Encuentre el polinomio generador `g(X)` **b)** Determine las capacidades del código si funciona como corrector y/o detector, encontrando la distancia mínima **c)** Especifique como combinación lineal de qué dígitos de información?, se obtiene cada chequeador. (mayores potencia de G a la derecha) **d)** Determine si el polinomio `R(x) = X⁸ + X⁴` es válido o no. Si no lo es corríjalo. **e)** Obtenga la capacidad de corrección a nivel de ráfaga si se entrelazan 20 códigos generados por `h(x)`.

Resolución — [Completado]

B) Control de error. Es exactamente el ejercicio de **2012 Tema 2** (código (14, 4)): ver 4.2 — `g(x) = 1 + x + x³ + x⁷ + x⁸ + x¹⁰`, `dmin = 6` (corrige 2 / detecta 5), chequeadores `C1 … C10` de 0.2, `R(x) = x⁸ + x⁴` no válido → error doble en `x⁴, x⁸` → corregida = palabra nula, ráfaga con 20 códigos = **40** bits.

A1) Claves para descifrar y texto en claro. Cifrado: Vigenère `KERBEROS` primero y transposición `4×5` clave `(2,5,3,1,4)` después → se deshace la transposición y luego Vigenère.

- Clave de **Vigenère de descifrado** (inverso aditivo de cada letra, `27 − k`): `K E R B E R O S` → `Q W J Z W J M I` (equivale a restar la clave de cifrado).
- Clave de **transposición inversa** (a qué columna del claro corresponde cada posición de lectura): `(2,5,3,1,4)` → `(4,1,3,5,2)`.
- Cuentas: 20 letras → 5 columnas de 4 letras; se parte el cifrado en `QVVÑ | MWHF | VYMQ | SVKÑ | TKAG`, se reubica cada bloque en su columna, se lee por filas (`MTVQSWKYVVHAMVKFGQÑÑ` con la convención "la columna `j` se lee en la posición k_j") y se resta `KERBEROS`.

[Nota] Con esta convención (la que dio texto legible en el parcial de 2012) el resultado es `CPEPOFVGMRPZIEVNWNWN`, que **no tiene sentido**. Se probaron ambas convenciones de lectura de la clave, ambos órdenes de aplicación (transposición/Vigenère), alfabetos de 26 y de 27 letras y corrimientos `±1`: ninguna combinación da texto en castellano ni en inglés. Lo más probable es que haya una **errata en el criptograma o en las claves del enunciado**; el procedimiento y las claves de descifrado son los indicados arriba. (Estos mismos resultados para Tema 2 en 5.2.)

A2) Confusión y difusión.

- **Confusión:** transformación sobre el texto en claro que mezcla sus elementos y complica la dependencia funcional entre la clave y el criptograma; se logra con **sustituciones** → en el ejercicio, **Vigenère** introduce confusión.
- **Difusión:** transformación que dispersa las propiedades estadísticas del lenguaje sobre todo el criptograma; se logra con **transposiciones** → en el ejercicio, la **transposición** introduce difusión.

A3) Entropías que modifica cada algoritmo. Vigenère (polialfabético): modifica H_o y las de orden superior. Transposición: **no** modifica H_o pero sí las de orden ≥ 1. En el ejercicio (resultado mecánico, convención "columna `j` en posición k_j"): claro `3,5464`, tras Vigenère `3,5219` (13 símbolos distintos, `N = 20`), cifrado `3,5219` (la transposición no cambia H_o).
¿Cuál es mejor? Bajo el criterio "modifica más niveles de entropía" el mejor es **Vigenère** (altera H_o, H_1, …); la transposición solo altera de orden 1 en adelante. Juntos se complementan: confusión + difusión.

5.2 Tema 2

> **A) CRIPTO:** Dado el siguiente texto cifrado: `F P N Q C M O A A J X S X G T Y H N F G O`, se sabe que fue cifrado con cifrador compuesto, utilizando los algoritmos: Vigenère con clave `K1 = (H,E,L,L,M,A,N)` y luego una transposición, utilizando una matriz de 3x7, con clave `K2 = (6,5,1,7,3,2,4)` (el texto en matriz fue dispuesto por filas). Encuentre las claves para descifrar el texto y el texto en claro. **b)** Identifique ventajas y desventajas de los métodos de cifrado simétrico y asimétrico. Cómo funciona un sistema de cifrado híbrido. **c)** Si se consideran A: Mensaje original y B Mensaje encriptado, cuánto debe valer la información mutua I(A,B) para que el sistema criptográfico se considere bueno. Justifique su respuesta.
>
> **CONTROL DE ERROR:** Sea el polinomio `h(x) = X³ + X² + X + 1` generador de un código (12, k, dmin) **a)** Encuentre la capacidad del código **b)** Encuentre la matriz generadora `G` y especifique como combinación lineal de qué dígitos de información se obtiene cada chequeador. **c)** Si se entrelazan 20 de los anteriores códigos qué nueva capacidad adquiere el código? **d)** Sea el siguiente código ISBN (con diez dígitos) encontrar el símbolo (?) `1−122−?3445−5`. ¿Cuál es la capacidad del código?

Resolución — [Completado]

A) Criptografía. Vigenère `HELLMAN` y luego transposición `3×7`, clave `(6,5,1,7,3,2,4)`.

- Clave de Vigenère de descifrado (`27 − k`): `H E L L M A N` → `T W P P O A Ñ`; clave de transposición inversa: `(6,5,1,7,3,2,4)` → `(3,6,5,7,2,1,4)`.
- Resultado mecánico con la convención de 2012 (columna `j` en la posición k_j): transposición inversa `YXFFOQJHGPGACXNTNOAMS`, Vigenère inverso `RTUUDQWACFVOCLGPCEOMG`.

[Nota] Igual que en el Tema 1 (5.1), ninguna convención da texto legible → casi seguro hay una errata en el enunciado. H_o: claro `3,8802`, tras Vigenère `3,8209`, cifrado `3,8209` (15 símbolos distintos de 21).

b) Simétrico vs. asimétrico; cifrado híbrido: ver PARTE 6.

c) `I(A,B)` debe ser **mínima, idealmente 0** (el criptograma no da información sobre el mensaje); un sistema perfecto "no debería tener relación" entre ambos, aunque en la práctica siempre existe alguna.

Control de error. Código **(12, 3)** (tabla 0.2).

- **a)** `k = 3`, $g(x) = x^9 + x^8 + x^5 + x^4 + x + 1$, `dmin = 6`: corrige 2 (y detecta 3 más) / detecta 5.
- **b)** `G` (3×12, sistemática) y chequeadores: `C1 = I1⊕I2⊕I3`, `C2 = I1`, `C3 = I2`, `C4 = I3`, `C5 = I1⊕I2⊕I3`, `C6 = I1`, `C7 = I2`, `C8 = I3`, `C9 = I1⊕I2⊕I3`:

```
G = 110011001 100
    101010101 010
```

```
     100110011 001
```

- **c)** Con 20 códigos entrelazados la capacidad de corrección de **ráfaga** pasa a `20·t = 20·2 =` **40** bits (se mantiene `t = 2` para errores aleatorios dentro de cada palabra).
- **d) ISBN** `1-122-?3445-5`**:** pesos `10…1`: `10 + 9 + 16 + 14 + 6x + 15 + 16 + 12 + 10 + 5 = 107 + 6x`; `107 mod 11 = 8` → `6x ≡ 3 (mod 11)` → `x = 3·2 = 6` → `1-122-63445-5` (suma `143 = 13·11`). **Capacidad:** `dmin = 2` → detecta 1 error y no corrige.

PARTE 6 — Preguntas teóricas con sus respuestas (`Preguntas Teóricas parcial 2 Teoría de la información.pdf`)

Respuestas de un alumno a las teóricas que se repiten en los parciales 2010–2013. Lo marcado **[Completado]** es una pregunta que el archivo dejaba sin responder; **[Corrección]** señala un error del alumno.

Criptografía

1. Orden decreciente según entropía relativa `*.exe`, `*.txt`, `*.rar`. El `.rar` va primero (la compresión aumenta significativamente la entropía), luego el `.exe` y por último el `.txt` (texto plano). → `.rar > .exe > .txt`.

2. Diferencias entre criptografía simétrica y asimétrica; ventajas y desventajas.

	Simétrico	**Asimétrico**
Claves	Una única clave (secreta) para cifrar y descifrar	Dos claves: la pública cifra y la privada descifra; cada extremo tiene su propio par
Uso	Generalmente para cifrar el mensaje	Generalmente para intercambio de clave y firma digital
Gestión de claves	No permite una gestión lógica y eficiente: hay que memorizar un gran número de claves	Solo hay que memorizar la clave privada; número de claves reducido
Longitud / espacio de claves	Del orden de la centena de bits (128 o más); para atacarlo hay que recorrer todo el espacio de claves	Del orden de los miles de bits (1024 o más); no es necesario atacar todo el espacio de claves
Vida de las claves	Muy corta (segundos o milisegundos), aleatoria; imposible de obtener en su tiempo de vida	Larga (meses o un año); son propias del usuario
Autenticación	Permite autenticar el mensaje pero no al emisor (al menos de forma sencilla); no tiene firma digital	Al haber clave pública y privada se logra autenticación del mensaje y del emisor (firma digital)
Velocidad	Muy alta (cientos de MB/s en hardware)	Muy lenta (cientos de KB/s en hardware)
Confidencialidad e integridad	Se logran protegiendo la clave en el cifrado y descifrado	Se logran por separado: firmando con la clave privada del emisor y cifrando con la clave pública del receptor

3. ¿Cómo funciona un sistema de cifrado híbrido? Usa criptografía simétrica y asimétrica (no es más ni menos seguro que cada una por separado: aporta características extras). El emisor cifra el mensaje con una **clave simétrica** (aprovechando su velocidad) y luego cifra esa clave simétrica con la **clave pública** del destinatario. Envía el paquete (mensaje cifrado con la simétrica + clave simétrica cifrada con la pública). El destinatario descifra la clave simétrica con su **clave privada** y con ella descifra el mensaje.

4. ¿Cuál fuente conviene comprimir: la original, la encriptada intermedia o la encriptada final? *(sin respuesta en el archivo)* **[Completado]** La **original**: la compresión aprovecha la redundancia (distribución desigual de símbolos) y el cifrado la destruye al aplanar las frecuencias; hay que **comprimir antes de cifrar** (ver el cálculo en 3, ejercicio 1.D).

5. ¿Cuánto debe valer la información mutua `I(A,B)`**, con** `A` **el mensaje original y** `B` **el encriptado? ¿Máxima o mínima?** Debe ser **mínima**: la relación entre los dígitos de entrada y los de salida debe ser mínima. En un sistema criptográfico perfecto no debería existir relación, es decir, `I(A,B)` debería ser 0, lo que en la práctica es imposible porque siempre existe alguna relación.

6. Necesidad de una base prima. Ofrece **unicidad** en el proceso de decodificación: no se generan ambigüedades (no pueden haber dos mensajes posibles ante uno recibido); con base prima hay una sola opción (todo no nulo tiene inverso).

7. Definir confusión y difusión. ¿Qué algoritmo introduce cada una? Confusión: transformación sobre el texto en claro con el objetivo de mezclar sus elementos, aumentando la complejidad de la dependencia funcional entre clave y criptograma; se logra con **sustituciones**. **Difusión:** transformación con el objeto de dispersar las propiedades estadísticas del lenguaje sobre todo el criptograma; se logra con **transposiciones**.

8. Niveles de entropía modificados por cada algoritmo. ¿Cuál es mejor? Las **transposiciones** modifican las entropías de orden 1 en adelante (no la de orden 0). Si la clave cambia en cada posición (como Vigenère), se altera la entropía de orden 0 en adelante. Entropía de orden 0: símbolos individuales; si una clave reemplaza siempre un símbolo por el mismo (la `A` siempre por la `C`), no se modifica H_o; si la primera vez lo reemplaza por un símbolo y la segunda vez por otro distinto, sí. **RSA, afín y César no modifican la entropía de orden 0** (sustitución monoalfabética); **Vigenère sí**, porque cada vez se reemplaza por un símbolo diferente. Entropía de orden superior (1,

2, …): cada símbolo depende de los anteriores (orden 1: del símbolo anterior; orden 2: de los dos anteriores, etc.).
[Corrección] Cuidado con la frase "RSA, afín y César no modifican la entropía de orden 0, pero sí las de orden superior": al ser relabelaciones biyectivas de símbolos tampoco alteran las de orden superior; lo que conserva H_0 y altera las superiores es la **transposición** (ver PARTE 1, ejercicio 5).

Extra (el alumno lo agrega como nota): un codificador óptimo usa el mínimo número de bits para codificar un mensaje: códigos cortos para los mensajes frecuentes y los más largos para los menos frecuentes, optimizando el rendimiento del canal o del almacenamiento.

Corrección de errores

1. Si se entrelazan n códigos, ¿qué nuevas capacidades adquiere el código? *(sin respuesta en el archivo)* **[Completado]** Adquiere capacidad de corrección de **ráfagas**: una ráfaga de hasta $n \cdot t$ bits consecutivos (los bits contiguos se reparten entre las n palabras, cada una con a lo sumo t errores); la capacidad contra errores aleatorios dentro de cada palabra sigue siendo t.

2. Propiedades del polinomio generador de un código cíclico. (I) $g(x)$ es de grado $n-k$; (II) $g(x)$ es factor de $x^n + 1$; (III) es el polinomio de menor grado y primo; (IV) $V(x)$ es múltiplo de $g(x)$ y todo múltiplo de $g(x)$ es palabra válida; (V) todo desplazamiento o rotación de $V(x)$ se transforma en otra $V(x)$.

3. Calcular el dígito faltante en un ISBN. *(sin desarrollo en el archivo)* **[Completado]** Ver 0.4 y los ejemplos resueltos: $3-540-?3218-9 \to 3$ (2020), $1-122-?3445-5 \to 6$ (2013 T2), $234323?323 \to 0$ (2012 T2).

4. Capacidad de un código ISBN. En general detectan **un único error** en una posición: un cambio en cualquier posición cambia necesariamente el valor del chequeador. Si se modifican al menos dos dígitos en distintas posiciones de manera que el chequeador resulte el mismo, el ISBN "es correcto" pero corresponde a otro libro: es un **error indetectable**. ($dmin = 2$.)

5. Cuadro con información de un campo de Galois y códigos Reed–Solomon. Tabla del alumno (se completan con la regla $n = q - 1$, $n - k = 2t$):

Capacidad	Grado $g(x)$ ($= 2t$)	Grado $f(x)$ (campo)	Tamaño campo q	Longitud $V(x)$ (n)	Bits por byte	Bytes de chequeo	Bytes de información
Corrector doble ($t = 2$)	4	5	32	31	5	4	27
Corrector triple ($t = 3$)	6	5	32	31	5	6	25
Corrector triple ($t = 3$)	6	4	16	15	4	6	9
Corrector de 7 bytes ($t = 7$)	14	8	256	255	8	14	241

6. Capacidad de corrección a nivel de ráfaga si se entrelazan n códigos generados por $h(x)$. Se suele construir códigos que corrigen errores aleatorios dentro de un bloque; al entrelazar códigos de corrección aleatoria se agrega capacidad de corrección en ráfaga. En el error aleatorio la probabilidad de error en cada bit es independiente de la de los bits vecinos; en una ráfaga los errores son consecutivos y tienen una causa común (por ejemplo, un rayón en un CD). Si un bloque corrige 10 errores y se entrelazan m bloques (se envía el primer bit del primer bloque, el primero del segundo, … hasta el primero del bloque m; luego el segundo de cada uno, etc.), la ráfaga corregible es la capacidad aleatoria por el número de códigos: $10 \cdot m$. (Fórmula general: $L \cdot t$.)

7. ¿Qué características tienen las matrices G y H entre sí? ¿Y qué significa? Si $G = [I, P]$ entonces $H = [P^T, I]$; si $G = [P, I]$ (información en las mayores potencias) entonces $H = [I, P^T]$. La característica en común es que son **ortogonales**: se hace el producto escalar del vector que se mira con una matriz del espacio perpendicular al de los vectores válidos, de tal manera que si el vector es válido el producto es 0 y si hay un error el resultado difiere de 0.
[Corrección] El alumno termina diciendo "si dos matrices son ortogonales, el producto de dichas matrices es la matriz identidad": **es la matriz nula**, no la identidad: $G \cdot H^T = 0$ (cada fila de G es ortogonal a cada fila de H).

8. ¿Un código cíclico con capacidad 10 qué capacidad tiene? Con $dmin = 10$ y $dmin \geq 2t + 1$: como **corrector** corrige hasta **4** errores y detecta **5** ($4 + 5 + 1 = 10$); como **detector puro** detecta hasta **9** ($dmin - 1$).

Extras (relación $dmin$ ↔ capacidades; el alumno la deja como nota): la cantidad de coeficientes (términos) de $g(x)$ determina un tope para $dmin$.

$dmin$	Como corrector	Como detector puro
1	nada	nada
2	nada	detecta 1
3	corrige 1	detecta 2
4	corrige 1 (y detecta 2)	detecta 3

`dmin`	Como corrector	Como detector puro
5	corrige 2	detecta 4
6	corrige 2 y detecta 3	detecta 5
10	corrige 4 y detecta 5	detecta 9
128	corrige 63 y detecta 64	detecta 127

PARTE 7 — Resumen de errores frecuentes (los marcados por el docente y los detectados al verificar)

1. **Pesos de las palabras de un cíclico mal calculados** (2025, 1.a): el `dmin` final salió bien, pero la tabla de pesos tenía 11 de 15 valores incorrectos (los reales son 6, 8 y 14). Conviene verificar con la cota " `dmin ≤` cantidad de términos de `g(x)` ".
2. **La `H` de un cíclico se arma con el polinomio recíproco de `h(x)`** (2025, 1.b): usar `h(x)` sin invertir da una `H` que no es ortogonal a las palabras del código (`G·Hᵀ ≠ 0`). Siempre comprobar `G·Hᵀ = 0` .
3. **Copiar mal los bits** (2025, 1.d): `1110001101000` (13 bits) en lugar de `11010001101000` (14 bits).
4. **Distinguir corrección simultánea de detección pura** (docente, 2020): con `dmin = 6` , "como corrector corrige 2 y detecta 3" (no "corrige 2 y detecta 5"; el 5 es solo si se usa como detector puro). Regla: `dmin ≥ t + s + 1` .
5. **Errores de suma en ISBN** (docente, 2020: sumó 113 en vez de 158 → dígito 6 en vez de 3). Controlar siempre con `Σ ≡ 0 (mod 11)` .
6. **Error triple en un corrector doble** (docente, 2020): el síndrome puede coincidir con dos combinaciones distintas de tres columnas de `H` ; el decodificador no corrige: **pide retransmisión** (no genera un tercer error).
7. **ISBN-13 "se puede corregir"** (2025): solo bajo el supuesto de que el dígito equivocado es el de control; el código detecta pero no localiza el error.
8. **Entropía de orden 0 vs. superior** (2025, ej. 5): la que conserva `H₀` y altera las superiores es la **transposición**, no la monoalfabética (esta conserva todos los órdenes).
9. **Orden inverso al descifrar un compuesto**: si "primero A y luego B", se deshace primero B. Probar la convención de la clave de transposición; la que da texto con sentido es la buena.
10. **Numeración `A = 1` (2010)**: con `A = 0` el descifrado de Vigenère sale corrido una letra.
11. **Capacidad de ráfaga con entrelazado**: `L·t` (no `L` ni `t`): 7·2 = 14, 20·2 = 40, 25·2 = 50, 20·3 = 60.
12. `G·Hᵀ` **da la matriz nula, no la identidad** (Preguntas teóricas, CE-7).
13. **El peso de `g(x)` no es una cota inferior de `dmin` , es una cota superior** (Contreras, 2025 1.a): `g(x)` es una palabra del código, así que `dmin ≤ w(g)` . Sirve para chequear, no para afirmar.
14. **Datos mal copiados y exponente de Euler mal puesto** (Contreras, 2025 afín): usó `a = 137` en vez de 139 y el exponente `ϕ(n)` en vez de `ϕ(n) − 1` , y obtuvo `167369` cuando el inverso es `107.914` . Verificar siempre `a·a⁻¹ ≡ 1 (mod n)` .
15. **Dejar el descifrado a medias** (Contreras, 2025 afín): el texto quedó en `…2025` , sin `, COMPREN ACCIONES DE YPF` .
16. `dmin` **del código rectangular con dos respuestas** (6 en la PARTE 1, 4 en la PARTE 1 bis): no hay resolución oficial; hay que **declarar qué regla se usa** (producto `d₁·d₂` , o rectangular con/sin bit esquina) y que la capacidad sea consistente con ese `dmin` .

PARTE 8 — Qué se descartó o no se pudo procesar

- `parciales viejos/Parcial1TIpdf (1).pdf` : es el enunciado del **Parcial 1** (Temas 1–4, fuente de vocales, canal BSC, teóricas); idéntico byte a byte a `Parcial 1/Parciales 1/Parcial1TIpdf_1.pdf` . Está en la carpeta equivocada; su contenido ya figura en `Parcial 1 - Transcripcion completa.md` .
- `PARCIAL2TI.doc.pdf` (2010) trae además una **encuesta de cátedra 2010** de 14 preguntas abiertas (anónima, sobre la cursada); no es parte del parcial y no se transcribe.
- Última página del PDF de la alumna Arce (2020): foto de un documento personal; **no se transcribe**.
- La `H` manuscrita de Arce (2020, 1.D) y el cuaderno completo están escaneados a mano; se transcribieron solo las partes legibles con certeza y el resto se reemplazó por el cálculo verificado.
- Los criptogramas de 2013 (Temas 1 y 2) no dan texto con sentido con ninguna convención probada (ver 5.1 y 5.2).
- `Parcial N2 de Teoria de la Informacion Tema 1 Contreras Matias.docx` (PARTE 1 bis) no está en la carpeta del Parcial 2 sino en Descargas: se transcribió de ahí. No trae imágenes ni matrices; los incisos 1.b a 1.e quedaron sin resolver en el original.